

debug

dimensione	bubble	insertion	selection	merge	quick	hybrid_quick	std::sort
4	7.464e-08	5.649e-08	6.761e-08	4.8782e-07	9.733e-08	6.464e-08	1.6021e-07
8	4.7496e-07	2.528e-07	3.8534e-07	2.28787e-06	5.5772e-07	2.6398e-07	3.7733e-07
16	1.64738e-06	4.7767e-07	1.03699e-06	2.78644e-06	7.2017e-07	4.8397e-07	8.6129e-07
32	4.51784e-06	1.4328e-06	2.75141e-06	6.1292e-06	1.69527e-06	1.51345e-06	2.22421e-06
64	1.62746e-05	5.07242e-06	8.82234e-06	1.30563e-05	4.12461e-06	5.29417e-06	4.88727e-06
128	6.72901e-05	1.88632e-05	3.07824e-05	2.88073e-05	9.46648e-06	1.01331e-05	1.89549e-05
256	0.000233167	8.75906e-05	0.000116618	9.00864e-05	2.04119e-05	2.3742e-05	2.38295e-05
512	0.000920013	0.000306561	0.000448917	0.000157487	4.84775e-05	5.41922e-05	5.43116e-05
1024	0.00351923	0.00135373	0.00163148	0.000317243	0.000104174	0.000132233	0.000121554
2048	0.0152499	0.00582023	0.00629563	0.000654793	0.000225645	0.000235861	0.000291893
4096	0.0668324	0.0231504	0.0252046	0.00235229	0.000815502	0.000655917	0.000779969
8192	0.299868	0.0929166	0.105659	0.00309227	0.00121146	0.00133911	0.00133098

release

dimensione	bubble	insertion	selection	merge	quick	hybrid_quick	std::sort
4	3.569e-08	2.299e-08	2.743e-08	9.875e-08	3.089e-08	2.487e-08	2.396e-08
8	1.4013e-07	6.777e-08	1.1257e-07	2.6951e-07	1.0493e-07	6.557e-08	7.058e-08
16	6.3299e-07	1.6902e-07	3.5537e-07	9.1435e-07	2.9667e-07	1.669e-07	1.4082e-07
32	2.31239e-06	4.2968e-07	1.00708e-06	1.51869e-06	6.8508e-07	3.7903e-07	5.0035e-07
64	8.55644e-06	9.3721e-07	2.63466e-06	3.65493e-06	1.47949e-06	1.05022e-06	1.21285e-06
128	3.07207e-05	2.79618e-06	7.7085e-06	8.34566e-06	3.2039e-06	2.33948e-06	2.74174e-06
256	0.000114323	9.43992e-06	2.37424e-05	1.56049e-05	1.02163e-05	7.77817e-06	9.50394e-06
512	0.000419423	3.44701e-05	8.14683e-05	3.22443e-05	1.44895e-05	1.21601e-05	1.29122e-05
1024	0.00159492	0.000157336	0.000271168	6.42693e-05	3.3606e-05	2.75068e-05	2.97006e-05
2048	0.0061432	0.000538217	0.00106537	0.000125931	7.02471e-05	6.15619e-05	6.63615e-05
4096	0.0249324	0.00209268	0.00423055	0.000319449	0.000168745	0.000140961	0.000157105
8192	0.104631	0.00826799	0.0156814	0.000639025	0.000331863	0.000290949	0.000316788

Testando i vari algoritmi su vettori di piccole dimensioni, ho notato subito che l'insertion sort è l'algoritmo quadratico più veloce.

La soglia in cui il quick sort inizia a convenire dipende dal compilatore: in modalità debug l'insertion sort vince fino alla dimensione 32, venendo superato alla dimensione 64; in modalità release l'insertion sort rimane nettamente superiore fino alla dimensione 256, venendo superato alla dimensione 512.

Su questa base ho implementato una versione ibrida del quick sort. L'algoritmo inizia a dividere il vettore normalmente, ma non appena la "fetta" da ordinare diventa più piccola di 256 elementi, invece di continuare con la ricorsione passa all'insertion sort. I risultati confermano che questa è la soluzione migliore perché si taglia via tutto il tempo perso per gestire le chiamate ricorsive sui pezzetti piccoli.

Guardando invece ai vettori più grandi (es. 8192 elementi), si vede chiaramente il limite degli algoritmi quadratici: in release il bubble sort ci mette circa 0.104 secondi, mentre il quick sort impiega appena 0.00033 secondi.